

Git / GitHub

Versionnement de projets



Sommaire



De quoi s'agit-il ?

01

Git

02

GitHub



Git

GitHub

Git



Loading Git State



La gestion de version

Pourquoi gérer
des versions ?

- Travailler à plusieurs sur un même projet.
- Revenir à une version antérieure d'un fichier ou du projet complet (en cas de bugs)
- Développer des fonctionnalités plus facilement



Concept et histoire

Un logiciel de gestion de versions permet de stocker un ensemble de fichiers en conservant la chronologie de toutes les modifications qui ont été effectuées dessus.

Quelques dates-clés :

90' : Création de CVS (système de versionning)

Début 2000 : Apache SVN (Subversion)

2005 : Création de GIT par Linus Torvald



Pourquoi Git ?

Les avantages

Décentralisé

permet de commencer à travailler tout de suite (pas de serveur requis).

Très rapide

parce que décentralisé

“Relativement” simple

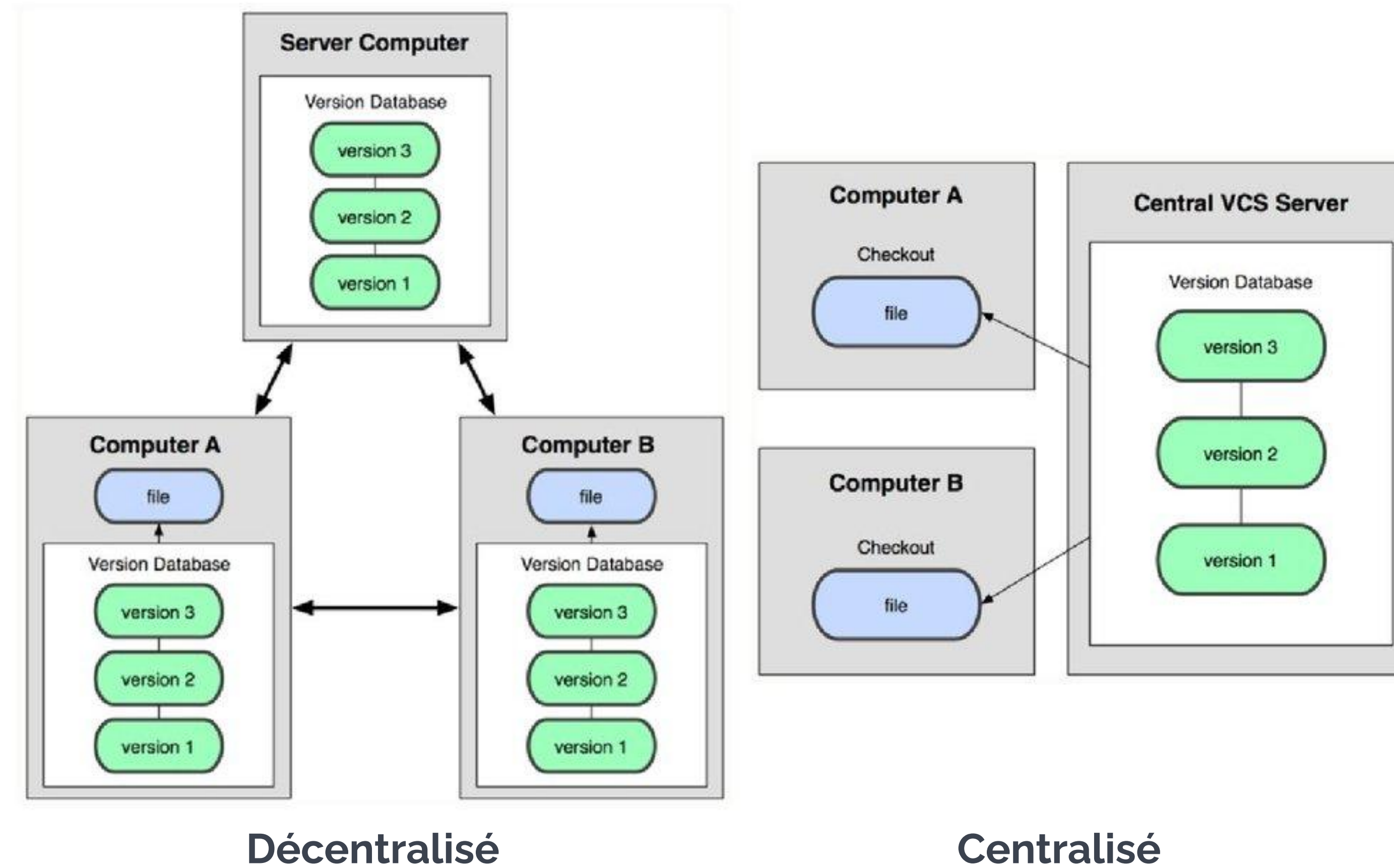
gestion des branches, création, fusion...

Mais il existe d'autres gestionnaires de versions:

- SVN (centralisé)
- CVS (centralisé)
- Mercurial (décentralisé)
- Bazaar (décentralisé)...



Centralisé VS Décentralisé





Base de fonctionnement

The 3 states

1. Tu fais une modification dans ton répertoire de travail
2. Tu ajoutes (**add**) tes fichiers à l'index (snapshots de tes fichiers)
3. Lorsque ton projet **est dans un état “propre”** (nouvelle fonctionnalité ou sous partie complète fonctionnelle), tu fais un **commit** qui va sauvegarder ton projet dans l'état où il se trouve dans ton index grâce à un *id* et un *commentaire*.

Concernant tes fichiers, Git gère 3 “arbres” principaux :

1. Répertoire de travail
2. Index (stage)
3. HEAD





Base de fonctionnement

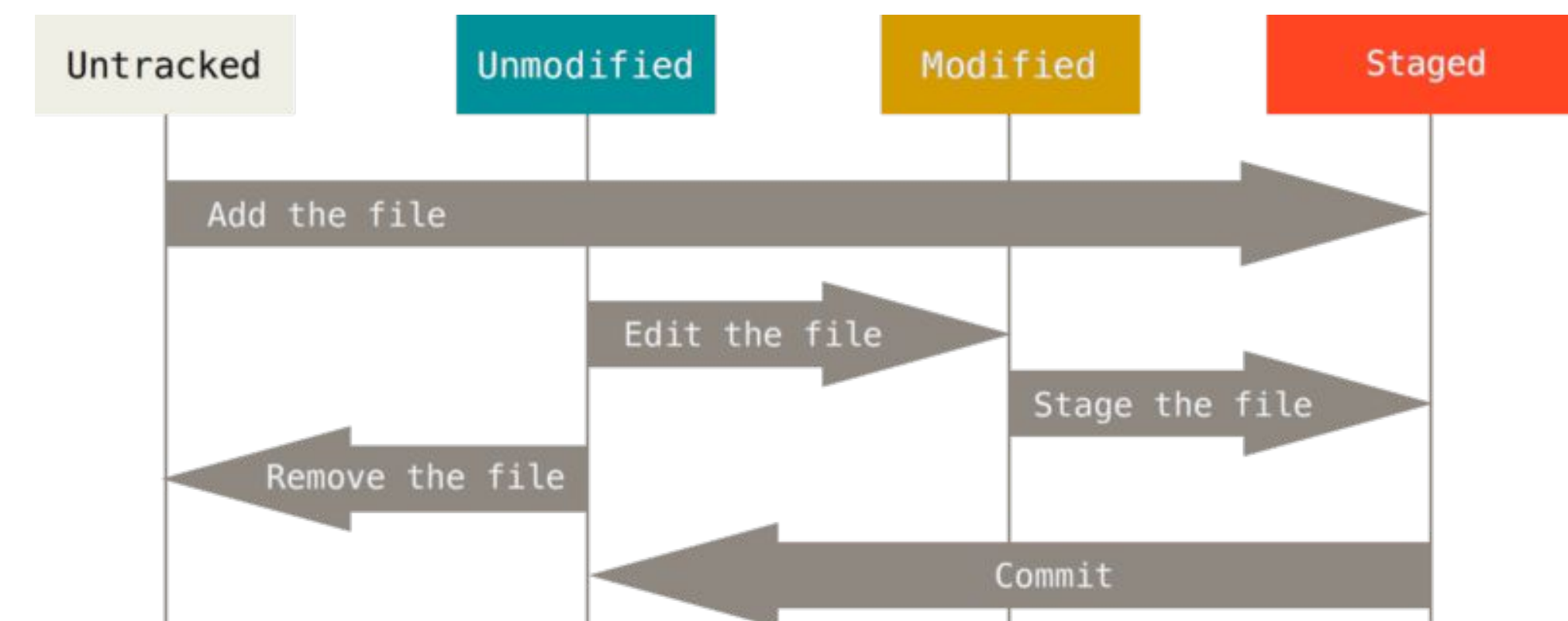
Status

Les fichiers dans le répertoire de travail ne peuvent avoir que 2 états :

1. Tracked
2. Untracked (non suivis par Git)

Les fichiers “tracked” peuvent avoir 3 états :

3. Modified (modifiés mais pas ajoutés à l’index)
4. Unmodified (commités)
5. Staged (modifiés et ajoutés à l’index)





Les commandes indispensables

Une commande très importante permet de connaître le statut actuel de ton repository :

> **git status**

- quels sont les fichiers qui n'ont **pas été indexés** (untracked)
- quels sont les fichiers modifiés ou supprimés et dont ces changements n'ont **pas été versionnés** (modified)
- quels sont les conflits



Les commandes indispensables

(suite) →

De plus, tu peux voir la liste des modifications effectuées précédemment grâce à :

> **git log**

Use et abuse de ces commandes !



Les commandes indispensables

(suite) →

- `init`
- `add`
- `commit`
- `status`
- `checkout`
- `branch`
- `push`
- `pull`
- `merge`
- `clone`
- `remote`
- `log`
- `diff`
- `stash`
- `reset`
- `revert`
- `config`
- `fetch`



Les branches

Définition

Par convention la branche **principale** d'un projet se nomme **main** .

(anciennement master).

Elle DOIT TOUJOURS être pleinement FONCTIONNELLE !

Ensuite, on distingue les **branches qui sont destinées à toujours exister**

(ex. : recette, dev, release, preprod...)

Et celles qui sont **amenées à disparaître** (ex. : bugfix, fonctionnalités temporaires, User Story...)



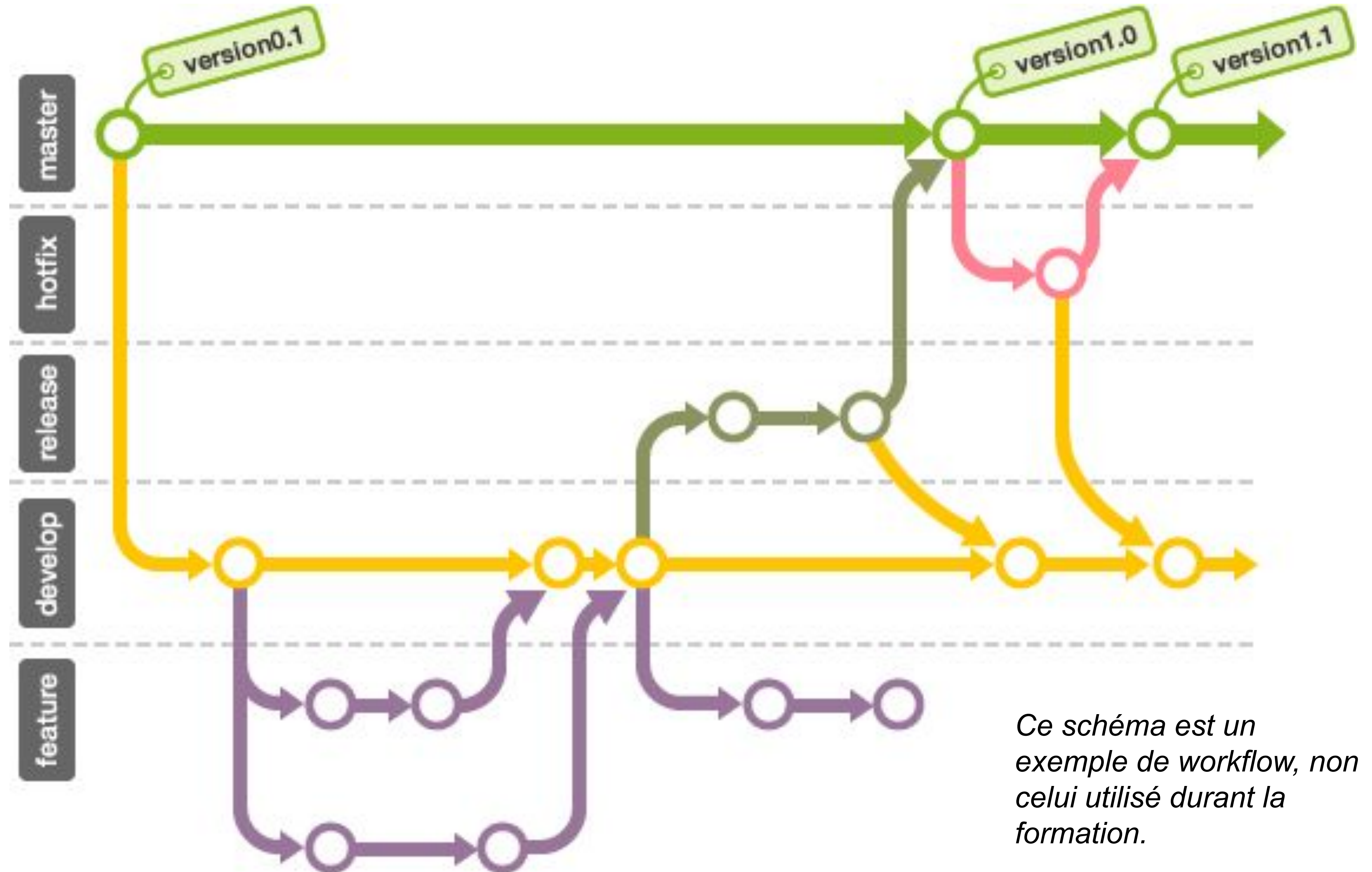
Git

GitHub

Workflow



Par où commencer ?



Ce schéma est un exemple de workflow, non celui utilisé durant la formation.



Git

GitHub

GitHub





GitHub, qu'est ce que c'est ?

Service web de gestion de version, créé en 2008

Basé sur Git

Modèle gratuit pour des dépôts publics/privés (open-source) et payant pour des fonctionnalités avancées

Attention à la confusion Git/GitHub ! Ça n'est **pas la même chose**.

Il existe d'autres services faisant la même chose :

- Bitbucket
- Gitlab
- ...



Fonctionnalités

Véritable réseau social des développeurs

- Suivi de projets
- Suivi de personnes
- Création d'équipes

Services pour les projets

- Wiki
- Page web
- Suivi de problèmes

Intégration de services externes



Git

GitHub

Fork



GitHub incite au **fork**, c'est à dire à copier un projet (open-source), à l'installer sur son propre compte GitHub et à le modifier selon ses besoins

Une **pull-request** (demande de contribution) peut être réalisée facilement, si besoin, vers le dépôt du contributeur initial



Conclusion

Avantages :

- le système de branches permet de basculer du dev d'une feature à une autre très facilement et sans interférence
- tout est décentralisé en local, pas besoin de connexion serveur pour gérer les fusions de branches
- algos de fusion très efficaces
- commits rapides

Contraintes :

- commandes peu intuitives au premier abord
- nécessité de s'entendre sur un workflow



Ressources

Git :

<https://git-scm.com/docs>

<http://rogerdudler.github.io/git-guide/index.fr.html>

<https://openclassrooms.com/courses/gerez-vos-codes-source-avec-git>

GitHub :

<https://openclassrooms.com/courses/gerer-son-code-avec-git-et-github/qu-est-ce-que-versionner-son-code>

Git workflows :

<http://www.nicoespeon.com/fr/2013/08/quel-git-workflow-pour-mon-projet/>

<https://makina-corpus.com/blog/metier/2014/un-workflow-git-efficace-pour-les-projets-a-moyen-long-terme>



MERCI

pour votre participation.

C'est à vous maintenant.

Des questions ?

Des remarques ?

